

关于复杂度分析的讨论

前言

一般来说，解决问题的方法不止一种。我们需要学习如何比较不同算法的性能，并选择最佳算法来解决特定的问题。一个算法的好坏，我们可以从时间和空间两个维度去衡量。执行时间越短，占用内存空间越小的算法，我们认为是更优的算法。并且，一般分为两个阶段，一是算法完成前的理论分析，二是算法完成后实际分析。

「理论分析」：这种算法的效率分析是通过假设所有其他因素，如处理器的速度等是恒定的，对算法的实现没有影响。

「实际分析」：当算法实现后，我们需要考虑算法采用编程语言，然后在特定计算机上执行该算法，其消耗的时间与计算机的硬件水平相关。在此分析中，我们要收集实际的统计数据，如运行时间和所需空间。

“好”算法的标准

解决一个问题的方法可能有很多，但能称得上算法的，首先它必须能彻底解决这个问题（称为准确性），且根据其编写出的程序在任何情况下都不能崩溃（称为健壮性）。

注意，程序和算法是完全不同的概念。算法是解决某个问题的想法、思路；而程序是在根据算法编写出来的真正可以运行的代码。例如，要依次输出一维数组中的数据元素的值，首先想到的是使用循环结构，在这个算法的基础上，我们才开始编写程序。

在满足准确性和健壮性的基础上，还有一个重要的筛选条件，即通过算法所编写出的程序的运行效率。程序的运行效率具体可以从 2 个方面衡量，分别为：

- 程序的运行时间。
- 程序运行所需内存空间的大小。

根据算法编写出的程序，运行时间更短，运行期间占用的内存更少，该算法的运行效率就更高，算法也就更好。

那么，如何衡量一个算法所编写出程序的运行效率呢？数据结构中，用时间复杂度来衡量程序运行时间的多少；用空间复杂度来衡量程序运行所需内存空间的大小。

为什么需要时间复杂度？

你可能会有些疑惑，我把代码跑一遍，通过统计、监控，就能得到算法执行的时间和占用的内存大小。为什么还要做时间、空间复杂度分析呢？这种分析方法能比我实实在在跑一遍得到的数据更准确吗？

首先，我可以肯定地说，你这种评估算法执行效率的方法是**正确**的。很多数据结构和算法书籍还给这种方法起了一个名字，叫**事后统计法**。但是，这种统计方法有**非常大的局限性**。

测试的结果依赖测试的环境

测试环境中硬件的不同会对测试结果有很大的影响。比如，我们拿同样一段代码，分别用 Intel Core i9 处理器和 Intel Core i3 处理器来运行，不用说，i9 处理器要比 i3 处理器执行的速度快很多。还有，比如原本在这台机器上 a 代码执行的速度比 b 代码要快，等我们换到另一台机器上时，可能会有截然相反的结果。

测试结果受数据规模的影响很大

如果测试数据规模太小，测试结果可能无法真实地反映算法的性能。比如，对于小规模的数据排序，插入排序可能反倒会比快速排序要快！

所以，我们需要一个不用具体的测试数据来测试，就可以粗略地估计算法的执行效率的方法。这就是我们今天要讲的时间、空间复杂度分析方法。

复杂度估计方法

通常，对于一个给定的算法，我们要做 两项分析。第一是从数学上证明算法的正确性，这一步主要用到形式化证明的方法及相关推理模式，如循环不变式、数学归纳法等。而在证明算法是正确的基础上，第二部就是分析算法的时间复杂度。算法的时间复杂度反映了程序执行时间随输入规模增长而增长的量级，在很大程度上能很好反映出算法的优劣与否。因此，作为程序员，掌握基本的算法时间复杂度分析方法是很有必要的。

算法执行时间需通过依据该算法编制的程序在计算机上运行时所消耗的时间来度量。而度量一个程序的执行时间通常有两种方法。

事后统计法

这种方法可行，但不是一个好的方法。该方法有两个缺陷：一是要想对设计的算法的运行性能进行评测，必须先依据算法编制相应的程序并实际运行；二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优势。

事前分析估算法

因事后统计方法更多的依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣。因此人们常常采用事前分析估算的方法。

在编写程序前，依据统计方法对算法进行估算。一个用高级语言编写的程序在计算机上运行时所消耗的时间取决于下列因素：

(1)．算法采用的策略、方法；(2)．编译产生的代码质量；(3)．问题的输入规模；(4)．机器执行指令的速度。

一个算法是由控制结构（顺序、分支和循环3种）和原操作（指固有数据类型的操作）构成的，则算法时间取决于两者的综合效果。为了便于比较同一个问题的不同算法，通常的做法是，从算法中选取一种对于所研究的问题（或算法类型）来说是基本操作的原操作，以该基本操作的重复执行的次数作为算法的时间量度。

时间复杂度

时间频度

一个算法执行所耗费的时间，从理论上是不能算出来的，必须上机运行测试才能知道。但我们不可能也没有必要对每个算法都上机测试，只需知道哪个算法花费的时间多，哪个算法花费的时间少就可以了。并且一个算法花费的时间与算法中语句的执行次数成正比例，哪个算法中语句执行次数多，它花费时间就多。一个算法中的语句执行次数称为语句频度或时间频度。记为 $T(n)$ 。

时间复杂度

在刚才提到的时间频度中， n 称为问题的规模，当 n 不断变化时，时间频度 $T(n)$ 也会不断变化。但有时我们想知道它变化时呈现什么规律。为此，我们引入时间复杂度概念。一般情况下，算法中基本操作重复执行的次数是问题规模 n 的某个函数，用 $T(n)$ 表示，若有某个辅助函数 $f(n)$ ，使得当 n 趋近于无穷大时， $T(n)/f(n)$ 的极限值为不等于零的常数，则称 $f(n)$ 是 $T(n)$ 的同数量级函数。记作 $T(n)=O(f(n))$ ，称 $O(f(n))$ 为算法的渐进时间复杂度，简称时间复杂度。

另外，上面公式中用到的 Landau 符号其实是由德国数论学家保罗·巴赫曼（Paul Bachmann）在其1892年的著作《解析数论》首先引入，由另一位德国数论学家艾德蒙·朗道（Edmund Landau）推广。Landau 符号的作用在于用简单的函数来描述复杂函数行为，给出一个上或下（确）界。在计算算法复杂度时一般只用到大O符号，Landau 符号体系中的小o符号、 Θ 符号等等比较不常用。这里的O，最初是用大写希腊字母，但现在都用大写英语字母O；小o符号也是用小写英语字母o， Θ 符号则维持大写希腊字母 Θ 。

$T(n) = O(f(n))$ 表示存在一个常数 C ，使得在当 n 趋于正无穷时总有 $T(n) \leq C * f(n)$ 。简单来说，就是 $T(n)$ 在 n 趋于正无穷时最大也就跟 $f(n)$ 差不多大。也就是说当 n 趋于正无穷时 $T(n)$ 的上界是 $C * f(n)$ 。其虽然对 $f(n)$ 没有规定，但是一般都是取尽可能简单的函数。例如， $O(2n^2+n+1) = O(3n^2+n+3) = O(7n^2+n) = O(n^2)$ ，一般都只用 $O(n^2)$ 表示就可以了。注意到大O符号里隐藏着一个常数 C ，所以 $f(n)$ 里一般不加系数。如果把 $T(n)$ 当做一棵树，那么 $O(f(n))$ 所表达的就是树干，只关心其中的主干，其他的细枝末节全都抛弃不管。

在各种不同算法中，若算法中语句执行次数为一个常数，则时间复杂度为 $O(1)$ ，另外，在时间频度不相同时，时间复杂度有可能相同，如 $T(n)=n^2+3n+4$ 与 $T(n)=4n^2+2n+1$ 它们的频度不同，但时间复杂度相同，都为 $O(n^2)$ 。按数量级递增排列，常见的时间复杂度有：常数阶 $O(1)$ ，对数阶 $O(\log_2 n)$ ，线性阶 $O(n)$ ，线性对数阶 $O(n \log_2 n)$ ，平方阶 $O(n^2)$ ，立方阶 $O(n^3)$ ，...， k 次方阶 $O(n^k)$ ，指数阶 $O(2^n)$ 。随着问题规模 n 的不断增大，上述时间复杂度不断增大，算法的执行效率越低。

常见的算法时间复杂度由小到大依次为： $O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < \dots < O(2^n) < O(n!)$

一般情况下，对一个问题（或一类算法）只需选择一种基本操作来讨论算法的时间复杂度即可，有时也需要同时考虑几种基本操作，甚至可以对不同的操作赋予不同的权值，以反映执行不同操作所需的相对时间，这种做法便于综合比较解决同一问题的两种完全不同的算法。

具体步骤

1、找出算法中的基本语句；算法中执行次数最多的那条语句就是基本语句，通常是最内层循环的循环体。

2、计算基本语句的执行次数的数量级；只需计算基本语句执行次数的数量级，这就意味着只要保证基本语句执行次数的函数中的最高次幂正确即可，可以忽略所有低次幂和最高次幂的系数。这样能够简化算法分析，并且使注意力集中在最重要的一点上：增长率。

3、用大O记号表示算法的时间性能。将基本语句执行次数的数量级放入大O记号中。

$O(1)$ 表示基本语句的执行次数是一个常数，一般来说，只要算法中不存在循环语句，其时间复杂度就是 $O(1)$ 。其中 $O(\log 2n)$ 、 $O(n)$ 、 $O(n \log 2n)$ 、 $O(n^2)$ 和 $O(n^3)$ 称为多项式时间，而 $O(2n)$ 和 $O(n!)$ 称为指数时间。计算机科学家普遍认为前者（即多项式时间复杂度的算法）是有效算法，把这类问题称为P（Polynomial,多项式）类问题，而把后者（即指数时间复杂度的算法）称为NP（Non-Deterministic Polynomial, 非确定多项式）问题。

一般来说多项式级的复杂度是可以接受的，很多问题都有多项式级的解——也就是说，这样的问题，对于一个规模是 n 的输入，在 n^k 的时间内得到结果，称为P问题。有些问题要复杂些，没有多项式时间的解，但是可以在多项式时间里验证某个猜测是不是正确。比如问4294967297是不是质数？如果要直接入手的话，那么要把小于4294967297的平方根的所有素数都拿出来，看看能不能整除。还好欧拉告诉我们，这个数等于641和6700417的乘积，不是素数，很好验证的，顺便麻烦转告费马他的猜想不成立。大数分解、Hamilton回路之类的问题，都是可以多项式时间内验证一个“解”是否正确，这类问题叫做NP问题。

分析法则

1、对于一些简单的输入输出语句或赋值语句,近似认为需要 $O(1)$ 时间

2、对于顺序结构,需要依次执行一系列语句所用的时间可采用大O下"求和法则"。求和法则:是指若算法的2个部分时间复杂度分别为 $T_1(n)=O(f(n))$ 和 $T_2(n)=O(g(n))$,则 $T_1(n)+T_2(n)=O(\max(f(n), g(n)))$ 。特别地,若 $T_1(m)=O(f(m))$, $T_2(n)=O(g(n))$,则 $T_1(m)+T_2(n)=O(f(m) + g(n))$

3、对于选择结构,如if语句,它的主要时间耗费是在执行then字句或else字句所用的时间,需注意的是检验条件也需要 $O(1)$ 时间

4、对于循环结构,循环语句的运行时间主要体现在多次迭代中执行循环体以及检验循环条件的时间耗费,一般可用大O下"乘法法则"。乘法法则: 是指若算法的2个部分时间复杂度分别为 $T_1(n)=O(f(n))$ 和 $T_2(n)=O(g(n))$,则 $T_1 * T_2=O(f(n) * g(n))$

5、对于复杂的算法,可以将它分成几个容易估算的部分,然后利用求和法则和乘法法则技术整个算法的时间复杂度。另外还有以下2个运算法则:(1) 若 $g(n)=O(f(n))$,则 $O(f(n))+ O(g(n))= O(f(n))$; (2) $O(Cf(n)) = O(f(n))$,其中C是一个正常数

例子分析

$O(1)$

```
Temp=i; i=j; j=temp;
```

以上三条单个语句的频度均为1，该程序段的执行时间是一个与问题规模 n 无关的常数。算法的时间复杂度为常数阶，记作 $T(n)=O(1)$ 。注意：如果算法的执行时间不随着问题规模 n 的增加而增长，即使算法中有上千条语句，其执行时间也不过是一个较大的常数。此类算法的时间复杂度是 $O(1)$ 。

$O(n^2)$

```
sum=0;                (一次)

for (i=1;i<=n;i++)    (n+1次)

for (j=1;j<=n;j++)    (n^2次)

sum++;                (n^2次)
```

因为 $\Theta(2n^2+n+1)=n^2$ (Θ 即：去低阶项，去掉常数项，去掉高阶项的常参得到)，所以 $T(n)=O(n^2)$;

```
for (i=1;i<n;i++)
{

y=y+1;                ①

for (j=0;j<=(2*n);j++)

x++;                ②

}
```

解：语句1的频度是 $n-1$. 语句2的频度是 $(n-1)*(2n+1)=2n^2-n-1$. $f(n)=2n^2-n-1+(n-1)=2n^2-2$;
又 $\Theta(2n^2-2)=n^2$ 该程序的时间复杂度 $T(n)=O(n^2)$.

一般情况下，对步进循环语句只需考虑循环体中语句的执行次数，忽略该语句中步长加1、终值判别、控制转移等成分，当有若干个循环语句时，算法的时间复杂度是由嵌套层数最多的循环语句中最内层语句的频度 $f(n)$ 决定的。

$O(n)$

```
a=0;
b=1;                ①
for (i=1;i<=n;i++) ②
{
s=a+b;            ③
b=a;              ④
a=s;              ⑤
}
```

解：语句1的频度：2,语句2的频度： n , 语句3的频度： $n-1$,语句4的频度： $n-1$,语句5的频度： $n-1$, $T(n)=2+n+3(n-1)=4n-1=O(n)$.

$O(\log_2 n)$

```
i=1;           ①
while (i<=n)
    i=i*2;      ②
```

解：语句1的频度是1, 设语句2的频度是 $f(n)$, 则： $2^{f(n)} \leq n; f(n) \leq \log_2 n$ 取最大值 $f(n) = \log_2 n$
 $T(n) = O(\log_2 n)$

$O(n^3)$

```
for(i=0; i<n; i++)
{
    for(j=0; j<i; j++)
    {
        for(k=0; k<j; k++)
            x=x+2;
    }
}
```

解：当 $i=m, j=k$ 的时候, 内层循环的次数为 k 当 $i=m$ 时, j 可以取 $0, 1, \dots, m-1$, 所以这里最内循环共进行了 $0+1+\dots+m-1 = (m-1)m/2$ 次所以, i 从0取到 n , 则循环共进行了: $0+(1-1)*1/2+\dots+(n-1)n/2 = n(n+1)(n-1)/6$ 所以时间复杂度为 $O(n^3)$.

一个经验规则：其中 c 是一个常量，如果一个算法的复杂度为 c 、 $\log_2 n$ 、 n 、 $n \cdot \log_2 n$, 那么这个算法时间效率比较高，如果是 $2n$ 、 $3n$ 、 $n!$ ，那么稍微大一些的 n 就会令这个算法不能动了，居于中间的几个则差强人意。算法时间复杂度分析是一个很重要的问题，任何一个程序员都应该熟练掌握其概念和基本方法，而且要善于从数学层面上探寻其本质，才能准确理解其内涵。

空间复杂度

类似于时间复杂度的讨论，一个算法的空间复杂度(Space Complexity) $S(n)$ 定义为该算法所耗费的存储空间，它也是问题规模 n 的函数。渐近空间复杂度也常常简称为空间复杂度。

空间复杂度(Space Complexity)是对一个算法在运行过程中临时占用存储空间大小的量度。一个算法在计算机存储器上所占用的存储空间，包括存储算法本身所占用的存储空间，算法的输入输出数据所占用的存储空间和算法在运行过程中临时占用的存储空间这三个方面。算法的输入输出数据所占用的存储空间是由要解决的问题决定的，是通过参数表由调用函数传递而来的，它不随本算法的不同而改变。存储算法本身所占用的存储空间与算法书写的长短成正比，要压缩这方面的存储空间，就必须编写出较短的算法。算法在运行过程中临时占用的存储空间随算法的不同而异，有的算法只需要占用少量的临时工作单元，而且不随问题规模的大小而改变，我们称这种算法是“就地”进行的，是节省存储的算法；有的算法需要占用的临时工作单元数与解决问题的规模 n 有关，它随着 n 的增大而增大，当 n 较大时，将占用较多的存储单元，例如快速排序和归并排序算法就属于这种情况。

如当一个算法的空间复杂度为一个常量，即不随被处理数据量 n 的大小而改变时，可表示为 $O(1)$ ；当一个算法的空间复杂度与以2为底的 n 的对数成正比时，可表示为 $O(\log_2 n)$ ；当一个算法的空间复杂度与 n 成线性比例关系时，可表示为 $O(n)$ 。若形参为数组，则只需要为它分配一个存储由实参传送来的一个地址指针的空间，即一个机器字长空间；若形参为引用方式，则也只需要为其分配存储一个地址的空间，

用它来存储对应实参变量的地址，以便由系统自动引用实参变量。

时间复杂度进一步划分

时间复杂度进一步可以划分为：最好、最坏、平均、摊均。直接看代码

```
// n表示数组array的长度
int find(int[] array, int n, int x) {
    int i = 0;
    int pos = -1;
    for (; i < n; ++i) {
        if (array[i] == x) {
            pos = i;
            break;
        }
    }
    return pos;
}
```

这段代码的功能非常简单，就是查找整数 x 在数组中的位置，找到就直接返回。因为，要查找的变量 x 可能出现在数组的任意位置。如果数组中第一个元素正好是要查找的变量 x ，那就不需要继续遍历剩下的 $n-1$ 个数据了，那时间复杂度就是 $O(1)$ 。但如果数组中不存在变量 x ，那我们就需要把整个数组都遍历一遍，时间复杂度就成了 $O(n)$ 。所以，不同的情况下，这段代码的时间复杂度是不一样的。

为了表示代码在不同情况下的不同时间复杂度，我们需要引入三个概念：最好情况时间复杂度、最坏情况时间复杂度和平均情况时间复杂度。

最好时间复杂度

最好情况时间复杂度就是，在最理想的情况下，执行这段代码的时间复杂度。在最理想的情况下，要查找的变量 x 正好是数组的第一个元素，这个时候对应的时间复杂度就是最好情况时间复杂度。

最坏时间复杂度

最坏情况时间复杂度就是，在最糟糕的情况下，执行这段代码的时间复杂度。如果数组中没有要查找的变量 x ，我们需要把整个数组都遍历一遍才行，所以这种最糟糕情况下对应的时间复杂度就是最坏情况时间复杂度。

平均情况时间复杂度

我们都知道，最好情况时间复杂度和最坏情况时间复杂度对应的都是极端情况下的代码复杂度，发生的概率其实并不大。为了更好地表示平均情况下的复杂度，我们需要引入另一个概念：平均情况时间复杂度，后面我简称为平均时间复杂度。

要查找的变量 x 在数组中的位置，有 $n+1$ 种情况：在数组的 $0\sim n-1$ 位置中和不在数组中。我们把每种情况下，查找需要遍历的元素个数累加起来，然后再除以 $n+1$ ，就可以得到需要遍历的元素个数的平均值，即：

$$\frac{1 + 2 + 3 + \dots + n + n}{n + 1} = \frac{n(n + 3)}{2(n + 1)}$$

我们知道，时间复杂度的大 O 标记法中，可以省略掉系数、低阶、常量，所以，咱们把刚刚这个公式简化之后，得到的平均时间复杂度就是 O(n)。

这个结论虽然是正确的，但是计算过程稍微有点儿问题。究竟是什么问题呢？我们刚讲的这 n+1 种情况，出现的概率并不是一样的。

我们知道，要查找的变量 x，要么在数组里，要么就不在数组里。这两种情况对应的概率统计起来很麻烦，为了方便你理解，我们假设在数组中与不在数组中的概率都为 1/2。另外，要查找的数据出现在 0~n-1 这 n 个位置的概率也是一样的，为 1/n。所以，根据概率乘法法则，要查找的数据出现在 0~n-1 中任意位置的概率就是 1/(2n)。

因此，前面的推导过程中存在的最大问题就是，没有将各种情况发生的概率考虑进去。如果我们把每种情况发生的概率也考虑进去，那平均时间复杂度的计算过程就变成了这样：

$$1 * 1 * \frac{1}{2n} + 2 * \frac{1}{2n} + 3 * \frac{1}{2n} + \dots + n * \frac{1}{2n} + n * \frac{1}{2} = \frac{3n + 1}{4}$$

这个值就是概率论中的加权平均值，也叫作期望值，所以平均时间复杂度的全称应该叫加权平均时间复杂度或者期望时间复杂度。

引入概率之后，前面那段代码的加权平均值为 (3n+1)/4。用大 O 表示法来表示，去掉系数和常量，这段代码的加权平均时间复杂度仍然是 O(n)。

实际上，在大多数情况下，我们并不需要区分最好、最坏、平均情况时间复杂度三种情况。像我们上一节课举的那些例子那样，很多时候，我们使用一个复杂度就可以满足需求了。只有同一块代码在不同的情况下，时间复杂度有量级的差距，我们才会使用这三种复杂度表示法来区分。

摊均时间复杂度

均摊时间复杂度，听起来跟平均时间复杂度有点儿像。对于初学者来说，这两个概念确实非常容易弄混。我前面说了，大部分情况下，我们并不需要区分最好、最坏、平均三种复杂度。平均复杂度只在某些特殊情况下才会用到，而均摊时间复杂度应用的场景比它更加特殊、更加有限。

```
// array表示一个长度为n的数组
// 代码中的array.length就等于n
int[] array = new int[n];
int count = 0;

void insert(int val) {
    if (count == array.length) {
        int sum = 0;
        for (int i = 0; i < array.length; ++i) {
            sum = sum + array[i];
        }
        array[0] = sum;
        count = 1;
    }
}
```

```
array[count] = val;
++count;
}
```

我先来解释一下这段代码。这段代码实现了一个往数组中插入数据的功能。当数组满了之后，也就是代码中的 `count == array.length` 时，我们用 `for` 循环遍历数组求和，并清空数组，将求和之后的 `sum` 值放到数组的第一个位置，然后再将新的数据插入。但如果数组一开始就有空闲空间，则直接将数据插入数组。

那这段代码的时间复杂度是多少呢？你可以先用我们刚讲到的三种时间复杂度的分析方法来分析一下。

最理想的情况下，数组中有空闲空间，我们只需要将数据插入到数组下标为 `count` 的位置就可以了，所以最好情况时间复杂度为 $O(1)$ 。最坏的情况下，数组中没有空闲空间了，我们需要先做一次数组的遍历求和，然后再将数据插入，所以最坏情况时间复杂度为 $O(n)$ 。

那平均时间复杂度是多少呢？答案是 $O(1)$ 。我们还是可以通过前面讲的概率论的方法来分析。

假设数组的长度是 n ，根据数据插入的位置的不同，我们可以分为 n 种情况，每种情况的时间复杂度是 $O(1)$ 。除此之外，还有一种“额外”的情况，就是在数组没有空闲空间时插入一个数据，这个时候的时间复杂度是 $O(n)$ 。而且，这 $n+1$ 种情况发生的概率一样，都是 $1/(n+1)$ 。所以，根据加权平均的计算方法，我们求得的平均时间复杂度就是：

$$1 * \frac{1}{n+1} + 1 * \frac{1}{n+1} + 1 * \frac{1}{n+1} + \dots + 1 * \frac{1}{n+1} + n * \frac{1}{n+1} = O(1)$$

至此为止，前面的最好、最坏、平均时间复杂度的计算，理解起来应该都没有问题。但是这个例子里的平均复杂度分析其实并不需要这么复杂，不需要引入概率论的知识。这是为什么呢？我们先来对比一下这个 `insert()` 的例子和前面那个 `find()` 的例子，你就会发现这两者有很大差别。

首先，`find()` 函数在极端情况下，复杂度才为 $O(1)$ 。但 `insert()` 在大部分情况下，时间复杂度都为 $O(1)$ 。只有个别情况下，复杂度才比较高，为 $O(n)$ 。这是 `insert()` 第一个区别于 `find()` 的地方。

我们再来看第二个不同的地方。对于 `insert()` 函数来说， $O(1)$ 时间复杂度的插入和 $O(n)$ 时间复杂度的插入，出现的频率是非常有规律的，而且有一定的前后时序关系，一般都是一个 $O(n)$ 插入之后，紧跟着 $n-1$ 个 $O(1)$ 的插入操作，循环往复。

所以，针对这样一种特殊场景的复杂度分析，我们并不需要像之前讲平均复杂度分析方法那样，找出所有的输入情况及相应发生概率，然后再计算加权平均值。

针对这种特殊的场景，我们引入了一种更加简单的分析方法：摊还分析法，通过摊还分析得到的时间复杂度我们起了一个名字，叫均摊时间复杂度。

那究竟如何使用摊还分析法来分析算法的均摊时间复杂度呢？

我们还是继续看在数组中插入数据的这个例子。每一次 $O(n)$ 的插入操作，都会跟着 $n-1$ 次 $O(1)$ 的插入操作，所以把耗时多的那次操作均摊到接下来的 $n-1$ 次耗时少的操作上，均摊下来，这一组连续的操作的均摊时间复杂度就是 $O(1)$ 。这就是均摊分析的大致思路。你都理解了吗？

均摊时间复杂度和摊还分析应用场景比较特殊，所以我们并不会经常用到。为了方便你理解、记忆，我这里简单总结一下它们的应用场景。如果你遇到了，知道是怎么回事儿就行了。

对一个数据结构进行一组连续操作中，大部分情况下时间复杂度都很低，只有个别情况下时间复杂度比较高，而且这些操作之间存在前后连贯的时序关系，这个时候，我们就可以将这一组操作放在一块儿分析，看是否能将较高时间复杂度那次操作的耗时，平摊到其他那些时间复杂度比较低的操作上。而且，在能够应用均摊时间复杂度分析的场所，一般均摊时间复杂度就等于最好情况时间复杂度。

尽管很多数据结构和算法书籍都花了很大力气来区分平均时间复杂度和均摊时间复杂度，但其实我个人认为，均摊时间复杂度就是一种特殊的平均时间复杂度，我们没必要花太多精力去区分它们。你最应该掌握的是它的分析方法，摊还分析。至于分析出来的结果是叫平均还是叫均摊，这只是个说法，并不重要。